

Object-Oriented Programming: Inheritance

Visual C# How to Program, 6/e

Introduction

- ▶ **Inheritance** allows a new class to absorb an existing class's members.
- ▶ A derived class normally adds its own fields and methods to represent a more specialized group of objects.
- ▶ Inheritance saves time by reusing proven and debugged high-quality software.

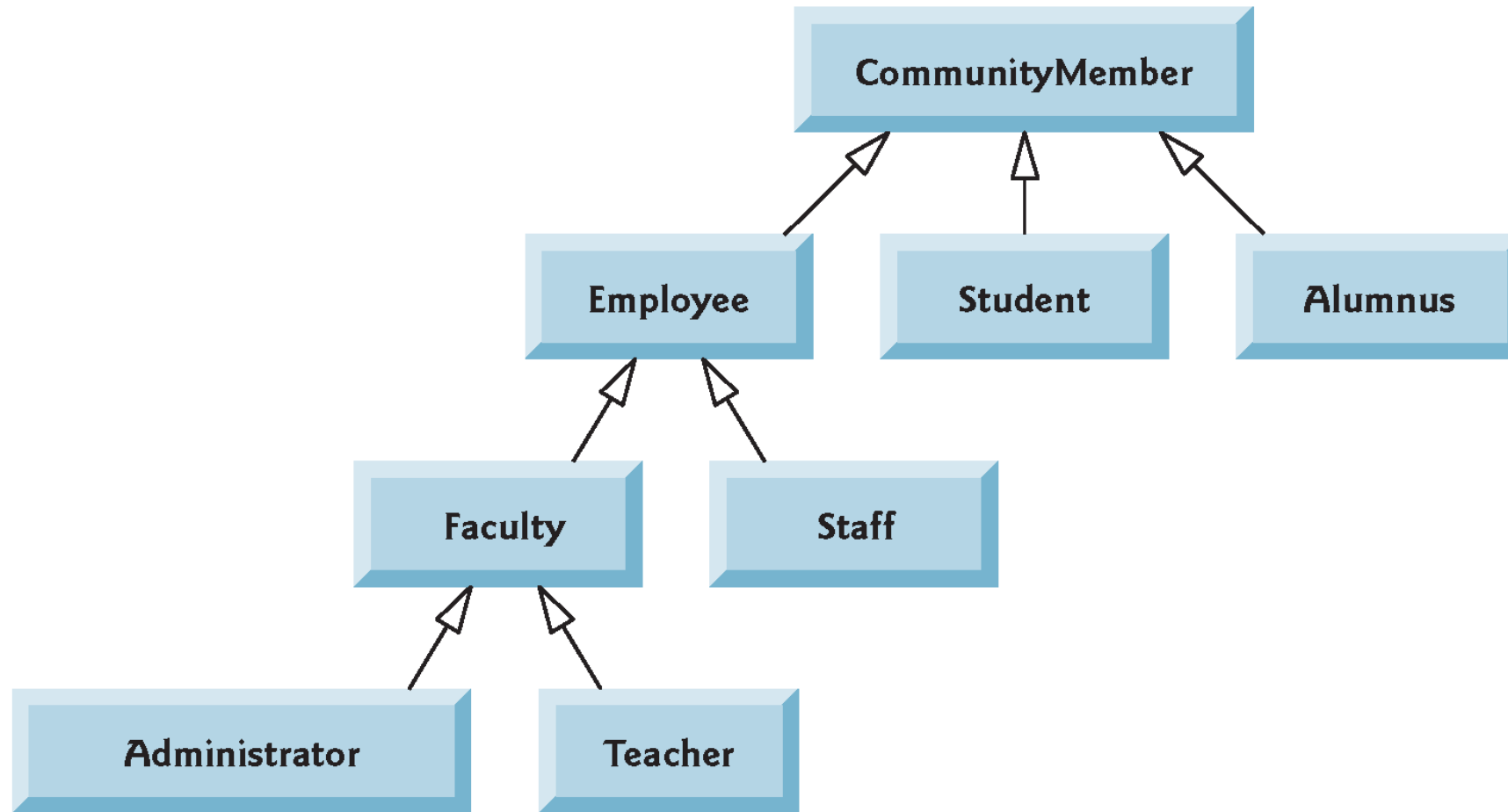
Introduction (Cont.)

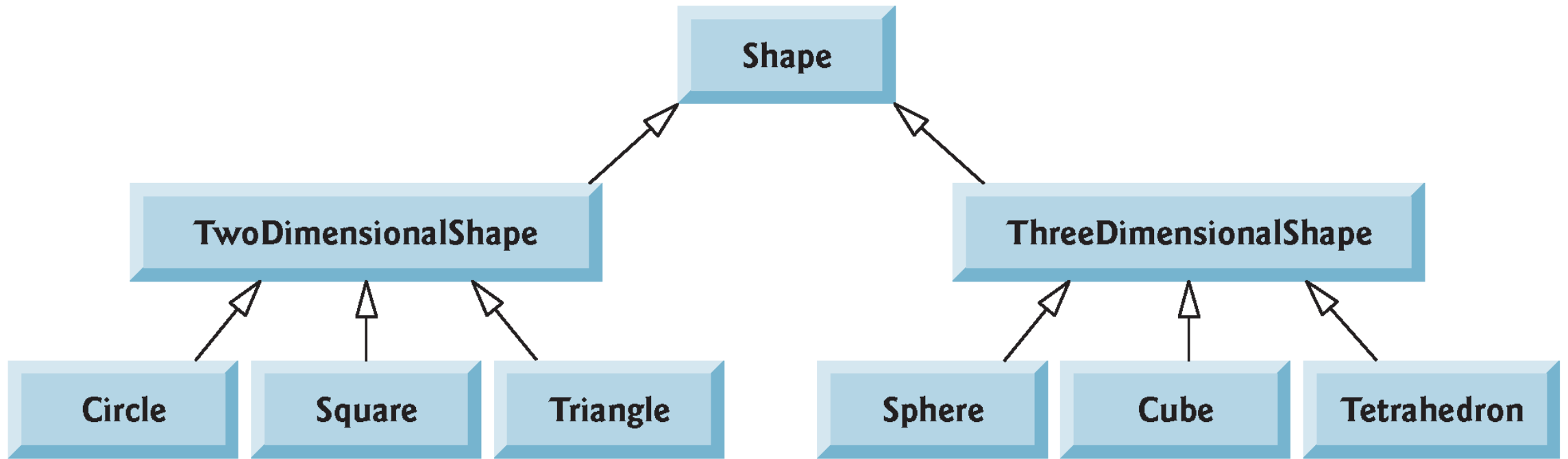
- ▶ The **direct base class** is the base class which the derived class explicitly inherits.
- ▶ An **indirect base class** is any class above the direct base class in the **class hierarchy**.
- ▶ The class hierarchy begins with class object.

Introduction (Cont.)

- ▶ The *is-a relationship* represents inheritance.
- ▶ For example, a car *is a* vehicle, and a truck *is a* vehicle.
- ▶ New classes can inherit from thousands of pre-built classes in **class libraries**.

Base class	Derived classes
Student	GraduateStudent, UndergraduateStudent
Shape	Circle, Triangle, Rectangle
Loan	CarLoan, HomeImprovementLoan, MortgageLoan
Employee	Faculty, Staff, HourlyWorker, CommissionWorker
SpaceObject	Star, Moon, Planet, FlyingSaucer
BankAccount	CheckingAccount, SavingsAccount





Base Classes and Derived Classes

- ▶ Objects of all classes that extend a common base class can be treated as objects of that base class.
- ▶ However, base-class objects cannot be treated as objects of their derived classes.
- ▶ A derived class can customize methods it inherits from its base class. In such cases, the derived class can **override** (redefine) the base-class method with an appropriate implementation.

protected Members

- ▶ A base class's `private` members *are* inherited by derived classes, but are *not* directly accessible by derived-class methods and properties.
- ▶ A base class's `protected` members can be accessed by members of that base class *and* by members of its derived classes, but not by clients of the class.

Creating and Using a `CommissionEmployee` Class

- ▶ `CommissionEmployee` represents an employee who is paid a percentage of their sales.

```
1 // Fig. 11.4: CommissionEmployee.cs
2 // CommissionEmployee class represents a commission employee.
3 using System;
4
5 public class CommissionEmployee : object
6 {
7     public string FirstName { get; }
8     public string LastName { get; }
9     public string SocialSecurityNumber { get; }
10    private decimal grossSales; // gross weekly sales
11    private decimal commissionRate; // commission percentage
12
```

```
13 // five-parameter constructor
14 public CommissionEmployee(string firstName, string lastName,
15     string socialSecurityNumber, decimal grossSales,
16     decimal commissionRate)
17 {
18     // implicit call to object constructor occurs here
19     FirstName = firstName;
20     LastName = lastName;
21     SocialSecurityNumber = socialSecurityNumber;
22     GrossSales = grossSales; // validates gross sales
23     CommissionRate = commissionRate; // validates commission rate
24 }
25
```

```
26 // property that gets and sets commission employee's gross sales
27 public decimal GrossSales
28 {
29     get
30     {
31         return grossSales;
32     }
33     set
34     {
35         if (value < 0) // validation
36         {
37             throw new ArgumentOutOfRangeException(nameof(value),
38                 value, $"{nameof(GrossSales)} must be >= 0");
39         }
40
41         grossSales = value;
42     }
43 }
44
```

```
45 // property that gets and sets commission employee's commission rate
46 public decimal CommissionRate
47 {
48     get
49     {
50         return commissionRate;
51     }
52     set
53     {
54         if (value <= 0 || value >= 1) // validation
55         {
56             throw new ArgumentOutOfRangeException(nameof(value),
57                 value, $"{nameof(CommissionRate)} must be > 0 and < 1");
58         }
59         commissionRate = value;
60     }
61 }
62 }
```

```
63
64 // calculate commission employee's pay
65 public decimal Earnings() => commissionRate * grossSales;
66
67 // return string representation of CommissionEmployee object
68 public override string ToString() =>
69     $"commission employee: {FirstName} {LastName}\n" +
70     $"social security number: {SocialSecurityNumber}\n" +
71     $"gross sales: {grossSales:C}\n" +
72     $"commission rate: {commissionRate:F2}";
73 }
```

Creating and Using a CommissionEmployee Class

- ▶ Declaring instance variables as `private` and providing `public` properties to manipulate and validate them helps enforce good software engineering.
- ▶ Constructors are *not* inherited.
- ▶ Either explicitly or implicitly, a call to the base-class constructor is made.
- ▶ Class object's default (empty) constructor does nothing.
- ▶ Even if a class does not have constructors, the default constructor will call the base class's default or parameterless constructor.

Creating and Using a CommissionEmployee Class (cont.)

CommissionEmployee Method ToString

- ▶ Method ToString is special—it's one of the methods that every class inherits directly or indirectly from class object.
- ▶ Method ToString returns a string representing an object.
- ▶ Class object's ToString method is primarily a placeholder that typically should be overridden by a derived class.
- ▶ To override a base-class method, a derived class must declare a method with keyword **override**.
- ▶ The method must have the same signature (method name, number of parameters and parameter types) and return type as the base-class method.

```
1  // Fig. 11.5: CommissionEmployeeTest.cs
2  // Testing class CommissionEmployee.
3  using System;
4
5  class CommissionEmployeeTest
6  {
7      static void Main()
8      {
9          // instantiate CommissionEmployee object
10         var employee = new CommissionEmployee("Sue", "Jones",
11         "222-22-2222", 10000.00M, .06M);
12
13         // display CommissionEmployee data
14         Console.WriteLine(
15             "Employee information obtained by properties and methods: \n");
16         Console.WriteLine($"First name is {employee.FirstName}");
```

```
17 Console.WriteLine($"Last name is {employee.LastName}");
18 Console.WriteLine(
19     $"Social security number is {employee.SocialSecurityNumber}");
20 Console.WriteLine($"Gross sales are {employee.GrossSales:C}");
21 Console.WriteLine(
22     $"Commission rate is {employee.CommissionRate:F2}");
23 Console.WriteLine($"Earnings are {employee.Earnings():C}");
24
25 employee.GrossSales = 5000.00M; // set gross sales
26 employee.CommissionRate = .1M; // set commission rate
27
28 Console.WriteLine(
29     "\nUpdated employee information obtained by ToString:\n");
30 Console.WriteLine(employee);
31 Console.WriteLine($"earnings: {employee.Earnings():C}");
32 }
33 }
```

Employee information obtained by properties and methods:

First name is Sue

Last name is Jones

Social security number is 222-22-2222

Gross sales are \$10,000.00

Commission rate is 0.06

Earnings are \$600.00

Updated employee information obtained by ToString:

commission employee: Sue Jones

social security number: 222-22-2222

gross sales: \$5,000.00

commission rate: 0.10

earnings: \$500.00

Creating a CommissionEmployee–BasePlusCommissionEmployee Inheritance Hierarchy

- ▶ Now we declare class BasePlusCommissionEmployee , which extends class CommissionEmployee

```
1  // Fig. 11.8: BasePlusCommissionEmployee.cs
2  // BasePlusCommissionEmployee inherits from CommissionEmployee.
3  using System;
4
5  public class BasePlusCommissionEmployee : CommissionEmployee
6  {
7      private decimal baseSalary; // base salary per week
8
9      // six-parameter derived-class constructor
10     // with call to base class CommissionEmployee constructor
11     public BasePlusCommissionEmployee(string firstName, string lastName,
12         string socialSecurityNumber, decimal grossSales,
13         decimal commissionRate, decimal baseSalary)
14         : base(firstName, lastName, socialSecurityNumber,
15             grossSales, commissionRate)
16     {
17         BaseSalary = baseSalary; // validates base salary
18     }
19
```

```
20    // property that gets and sets
21    // BasePlusCommissionEmployee's base salary
22    public decimal BaseSalary
23    {
24        get
25        {
26            return baseSalary;
27        }
28        set
29        {
30            if (value < 0) // validation
31            {
32                throw new ArgumentOutOfRangeException(nameof(value),
33                    value, $"{nameof(BaseSalary)} must be >= 0");
34            }
35
36            baseSalary = value;
37        }
38    }
39
```

```
40 // calculate earnings
41 public override decimal Earnings() =>
42     baseSalary + (commissionRate * grossSales);
43
44 // return string representation of BasePlusCommissionEmployee
45 public override string ToString() =>
46     // not allowed: attempts to access private base-class members
47     $"base-salaried commission employee: {FirstName} {LastName}\n" +
48     $"social security number: {SocialSecurityNumber}\n" +
49     $"gross sales: {grossSales:C}\n" +
50     $"commission rate: {commissionRate:F2}\n" +
51     $"base salary: {baseSalary}";
52 }
```

Error List				
Entire Solution		6 Errors	0 Warnings	0 Messages
	Build + IntelliSense	Search Error List		
	Code	Description	File	Line
	CS0506	'BasePlusCommissionEmployee.Earnings()': cannot override inherited member 'CommissionEmployee.Earnings()' because it is not marked virtual, abstract, or override	BasePlusCommissionEmployee.cs	41
	CS0122	'CommissionEmployee.commissionRate' is inaccessible due to its protection level	BasePlusCommissionEmployee.cs	42
	CS0122	'CommissionEmployee.commissionRate' is inaccessible due to its protection level	BasePlusCommissionEmployee.cs	50
	CS0122	'CommissionEmployee.grossSales' is inaccessible due to its protection level	BasePlusCommissionEmployee.cs	42
	CS0122	'CommissionEmployee.grossSales' is inaccessible due to its protection level	BasePlusCommissionEmployee.cs	49

Creating a CommissionEmployee–BasePlusCommissionEmployee Inheritance Hierarchy (cont.)

- ▶ A BasePlusCommissionEmployee object *is a* CommissionEmployee.
- ▶ Each derived-class constructor *must* implicitly or explicitly call its base-class constructor to ensure that the instance variables inherited from the base class are initialized properly.
- ▶ A constructor initializer with keyword **base** invokes the base-class constructor.

Creating a CommissionEmployee–BasePlusCommissionEmployee Inheritance Hierarchy (cont.)

BasePlusCommissionEmployee Method Earnings

- ▶ The **virtual** and **abstract** keywords indicate that a base-class method can be overridden in derived classes.
- ▶ The **override** modifier declares that a derived-class method overrides a **virtual** or **abstract** base-class method.
- ▶ This modifier also implicitly declares the derived-class method **virtual**.
- ▶ We need to declare CommissionEmployee's Earnings method **virtual**.

Creating a CommissionEmployee–BasePlusCommissionEmployee Inheritance Hierarchy (cont.)

- ▶ The compiler generates additional errors because base class `CommissionEmployee`'s instance variables are `private`.
- ▶ The errors can be prevented by using the `public` properties inherited from class `CommissionEmployee`.

```
1  // Fig. 11.9: BasePlusCommissionEmployee.cs
2  // BasePlusCommissionEmployee inherits from CommissionEmployee and has
3  // access to CommissionEmployee's protected members.
4  using System;
5
6  public class BasePlusCommissionEmployee : CommissionEmployee
7  {
8      private decimal baseSalary; // base salary per week
9
10     // six-parameter derived-class constructor
11     // with call to base class CommissionEmployee constructor
12     public BasePlusCommissionEmployee(string firstName, string lastName,
13         string socialSecurityNumber, decimal grossSales,
14         decimal commissionRate, decimal baseSalary)
15         : base(firstName, lastName, socialSecurityNumber,
16             grossSales, commissionRate)
17     {
18         BaseSalary = baseSalary; // validates base salary
19     }
20
```

```
21 // property that gets and sets
22 // BasePlusCommissionEmployee's base salary
23 public decimal BaseSalary
24 {
25     get
26     {
27         return baseSalary;
28     }
29     set
30     {
31         if (value < 0) // validation
32         {
33             throw new ArgumentOutOfRangeException(nameof(value),
34                 value, $"{nameof(BaseSalary)} must be >= 0");
35         }
36
37         baseSalary = value;
38     }
39 }
40
```

```
41 // calculate earnings
42 public override decimal Earnings() =>
43     baseSalary + (commissionRate * grossSales);
44
45 // return string representation of BasePlusCommissionEmployee
46 public override string ToString() =>
47     $"base-salaried commission employee: {FirstName} {LastName}\n" +
48     $"social security number: {SocialSecurityNumber}\n" +
49     $"gross sales: {grossSales:C}\n" +
50     $"commission rate: {commissionRate:F2}\n" +
51     $"base salary: {baseSalary}";
52 }
```

CommissionEmployee–BasePlusCommissionEmployee Inheritance Hierarchy Using private Instance Variables (cont.)

- ▶ Class BasePlusCommissionEmployee inherits from CommissionEmployee's and can access the private base-class members via the inherited public properties.

```
1  // Fig. 11.11: BasePlusCommissionEmployee.cs
2  // BasePlusCommissionEmployee inherits from CommissionEmployee and has
3  // controlled access to CommissionEmployee's private data via
4  // its public properties.
5  using System;
6
7  public class BasePlusCommissionEmployee : CommissionEmployee
8  {
9      private decimal baseSalary; // base salary per week
10
11     // six-parameter derived-class constructor
12     // with call to base class CommissionEmployee constructor
13     public BasePlusCommissionEmployee(string firstName, string lastName,
14         string socialSecurityNumber, decimal grossSales,
15         decimal commissionRate, decimal baseSalary)
16         : base(firstName, lastName, socialSecurityNumber,
17             grossSales, commissionRate)
18     {
19         BaseSalary = baseSalary; // validates base salary
20     }
```

```
21
22     // property that gets and sets
23     // BasePlusCommissionEmployee's base salary
24     public decimal BaseSalary
25     {
26         get
27         {
28             return baseSalary;
29         }
30         set
31         {
32             if (value < 0) // validation
33             {
34                 throw new ArgumentOutOfRangeException(nameof(value),
35                     value, $"{nameof(BaseSalary)} must be >= 0");
36             }
37
38             baseSalary = value;
39         }
40     }
```

```
41
42 // calculate earnings
43 public override decimal Earnings() => BaseSalary + base.Earnings();
44
45 // return string representation of BasePlusCommissionEmployee
46 public override string ToString() =>
47     $"base-salaried {base.ToString()}\nbase salary: {BaseSalary:C}";
48 }
```

11.4.5 CommissionEmployee–BasePlusCommissionEmployee Inheritance Hierarchy Using private Instance Variables (cont.)

- ▶ Class BasePlusCommissionEmployee's Earnings overrides class CommissionEmployee's Earnings method.
- ▶ The new version obtains the portion of the employee's earnings based on commission alone by calling CommissionEmployee's Earnings method with the expression `base.Earnings()`
- ▶ This method invocation is a good software engineering practice—by having BasePlusCommissionEmployee's Earnings method invoke CommissionEmployee's Earnings method to calculate part of a BasePlusCommissionEmployee object's earnings, we avoid duplicating the code and reduce code-maintenance problems

CommissionEmployee–BasePlusCommissionEmployee Inheritance Hierarchy Using private Instance Variables (Cont.)

Class BasePlusCommissionEmployeeTest

- ▶ Class BasePlusCommissionEmployeeTest performs the same manipulations on a BasePlusCommissionEmployee object and produces the same output, so we do not show it here.

Constructors in Derived Classes

- The derived-class constructor, before performing its own tasks, invokes its direct base class's constructor.
- This is done either explicitly or implicitly.
- If the base class is derived from another class, the base-class constructor invokes the constructor of the next class up in the hierarchy, and so on.

Software Engineering with Inheritance

- When a new class extends an existing class, the new class inherits the members of the existing class.
- We can customize the new class to meet our needs by including additional members and by overriding base-class members.
- Independent software vendors (ISVs) can develop and sell proprietary classes.
- Users then can derive new classes from these library classes without accessing the source code.

Software Engineering with Inheritance (Cont.)

- ▶ Effective software reuse improves the software-development process.
- ▶ Object-oriented programming facilitates software reuse, potentially shortening development time.
- ▶ The availability of substantial and useful class libraries helps deliver the maximum benefits of software reuse through inheritance.

Class object

- ▶ All classes inherit directly or indirectly from the object class.

Method	Description
<code>Equals</code>	This method compares the current object to another object for equality and returns <code>true</code> if they're equal and <code>false</code> otherwise. It takes any object as an argument. When objects of a particular class must be compared for equality, the class should override method <code>Equals</code> to compare the <i>contents</i> of the two objects. The website http://bit.ly/Overriding-EqualsCSharp explains the requirements for a properly overridden <code>Equals</code> method.

Method	Description
<code>Finalize</code>	This method cannot be explicitly declared or called. When a class contains a destructor, the compiler implicitly renames it to override the protected method <code>Finalize</code> , which is called <i>only</i> by the garbage collector before it reclaims an object's memory. The garbage collector is not guaranteed to reclaim an object, thus it's <i>not</i> guaranteed that an object's <code>Finalize</code> method will execute. When a derived class's <code>Finalize</code> method executes, it performs its task, then invokes the base class's <code>Finalize</code> method. In general, you should avoid using <code>Finalize</code> .

Method	Description
GetHashCode	A hashtable data structure relates objects, called <i>keys</i> , to corresponding objects, called <i>values</i> . We discuss <code>Hashtable</code> in Chapter 21. When a value is initially inserted in a hashtable, the key's <code>GetHashCode</code> method is called. The value returned is used by the hashtable to determine the location at which to insert the corresponding value. The key's hashcode is also used by the hashtable to locate the key's corresponding value.
GetType	Every object knows its own type at execution time. Method <code>GetType</code> (used in Section 12.5) returns an object of class <code>Type</code> (namespace <code>System</code>) that contains information about the object's type, such as its class name (obtained from <code>Type</code> property <code>FullName</code>).

Method	Description
<code>MemberwiseClone</code>	This protected method, which takes no arguments and returns an <code>object</code> reference, makes a copy of the object on which it's called. The implementation of this method performs a shallow copy —instance-variable values in one object are copied into another object of the same type. For reference types, only the references are copied.
<code>ReferenceEquals</code>	This <code>static</code> method receives two <code>object</code> references and returns <code>true</code> if they're the same instance or if they're <code>null</code> references. Otherwise, it returns <code>false</code> .
<code>ToString</code>	This method (introduced in Section 7.4) returns a <code>string</code> representation of the current object. The default implementation of this method returns the namespace followed by a dot and the class name of the object's class.